```
E0124093
B.Anusri
```

Computer Vision Face Detection Lab - Student Notebook

In this lab, you will use Amazon Rekognition to perform a facial detection of a known face.

The high-level steps you will perform in this lab are:

1. Create a collection.
2. Upload an image of a face that you want to detect to the Amazon SageMaker notebook.
3. Add the image to the collection.
4. View the bounding box that was created for the image.
5. List the faces in the collection.
6. Use the collection to find a face.
7. View the bounding box of the face that was found.
8. Delete the collection.

```
!pip install scikit-image
!pip install matplotlib
from skimage import io
from skimage.transform import rescale
from matplotlib import pyplot as plt

import boto3

import numpy as np

from PIL import Image, ImageDraw, ImageColor, ImageOps
```

```
Requirement already satisfied: scikit-image in
/home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages
(0.25.2)
Requirement already satisfied: numpy>=1.24 in
/home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages
(from scikit-image) (2.2.6)
Requirement already satisfied: scipy>=1.11.4 in
```

```
/home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages
(from scikit-image) (1.15.3)
Requirement already satisfied: networkx>=3.0 in
/home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages
(from scikit-image) (3.4.2)
Requirement already satisfied: pillow>=10.1 in
/home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages
(from scikit-image) (12.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /home/ec2-
user/anaconda3/envs/python3/lib/python3.10/site-packages (from scikit-
image) (2.38.0)
Requirement already satisfied: tifffile>=2022.8.12 in
/home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages
(from scikit-image) (2025.5.10)
Requirement already satisfied: packaging>=21 in
/home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages
(from scikit-image) (24.2)
Requirement already satisfied: lazy-loader>=0.4 in
/home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages
(from scikit-image) (0.6)
Collecting matplotlib
  Using cached matplotlib-3.10.9-cp310-cp310-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (52 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Using cached contourpy-1.3.2-cp310-cp310-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib)
  Using cached cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Using cached fonttools-4.65.0-cp310-cp310-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (125 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Using cached kiwisolver-1.5.1-cp310-cp310-
manylinux_2_12_x86_64.manylinux2010_x86_64.whl.metadata (5.2 kB)
Requirement already satisfied: numpy>=1.23 in
/home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages
(from matplotlib) (2.2.6)
Requirement already satisfied: packaging>=20.0 in
/home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages
(from matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in
/home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages
(from matplotlib) (12.3.0)
Collecting pyparsing>=3 (from matplotlib)
  Using cached pyparsing-3.3.3-py3-none-any.whl.metadata (5.9 kB)
Requirement already satisfied: python-dateutil>=2.7 in /home/ec2-
user/anaconda3/envs/python3/lib/python3.10/site-packages (from
matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in
```

```
/home/ec2-user/anaconda3/envs/python3/lib/python3.10/site-packages
(from python-dateutil>=2.7->matplotlib) (1.17.0)
Using cached matplotlib-3.10.9-cp310-cp310-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (8.8 MB)
Using cached contourpy-1.3.2-cp310-cp310-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (325 kB)
Using cached cycler-0.12.1-py3-none-any.whl (8.3 kB)
Using cached fonttools-4.65.0-cp310-cp310-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl (5.3 MB)
Using cached kiwisolver-1.5.1-cp310-cp310-
manylinux_2_12_x86_64.manylinux2010_x86_64.whl (1.6 MB)
Using cached pyparsing-3.3.3-py3-none-any.whl (126 kB)
Installing collected packages: pyparsing, kiwisolver, fonttools,
cycler, contourpy, matplotlib
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6/6 [matplotlib]6
[matplotlib]
atplotlib-3.10.9 pyparsing-3.3.3
```

Task 1: Creating a collection

In this task, you create a collection in Amazon Rekognition.

You only need to run this step once.

```
client = boto3.client('rekognition')
collection_id = 'Collection'
response = client.create_collection(CollectionId=collection_id)
print('Collection ARN: ' + response['CollectionArn'])
print('Status Code:' + str(response['StatusCode']))
print('Done...')
```

```
Collection ARN:
aws:rekognition:us-east-1:274956312578:collection/Collection
Status Code:200
Done...
```

Task 2: Uploading an image to search

```
filename = "mum.jpg"

faceimage = io.imread(filename)

plt.imshow(faceimage)
```

```
<matplotlib.image.AxesImage at 0x7f5f829e5db0>
```



Task 3: Adding the image to the collection

Add the image to the collection that you created earlier.

```
externalimageid = filename

with open(filename, 'rb') as fimage:
    response = client.index_faces(CollectionId = collection_id,
                                  Image={'Bytes': fimage.read()},
                                  ExternalImageId=externalimageid,
                                  MaxFaces=1,
                                  QualityFilter="AUTO",
                                  DetectionAttributes=['ALL'])

print('Results for ' + filename)
print('Faces indexed:')
for faceRecord in response['FaceRecords']:
     print('  Face ID: ' + faceRecord['Face']['FaceId'])
     print('  Location: {}'.format(faceRecord['Face']['BoundingBox']))

print('Faces not indexed:')
for unindexedFace in response['UnindexedFaces']:
    print(' Location: {}'.format(unindexedFace['FaceDetail']
['BoundingBox']))
    print(' Reasons:')
```

```
    for reason in unindexedFace['Reasons']:
        print('   ' + reason)
```

```
Results for mum.jpg
Faces indexed:
  Face ID: 51c54066-f5ea-4cff-ac3e-441e915275ff
  Location: {'Width': 0.3483645021915436, 'Height':
0.35312092304229736, 'Left': 0.30951350927352905, 'Top':
0.11020112037658691}
Faces not indexed:
```

Task 4: Viewing the bounding box for the detected face

If a face was found, the results should include the location of the face that was detected. Examine the bounding box on the image.

To do this, use the PIL library, which you imported earlier in this lab. By extracting the BoundingBox, you can draw a set of lines around the image.

```
img = Image.open(filename)
imgWidth, imgHeight = img.size

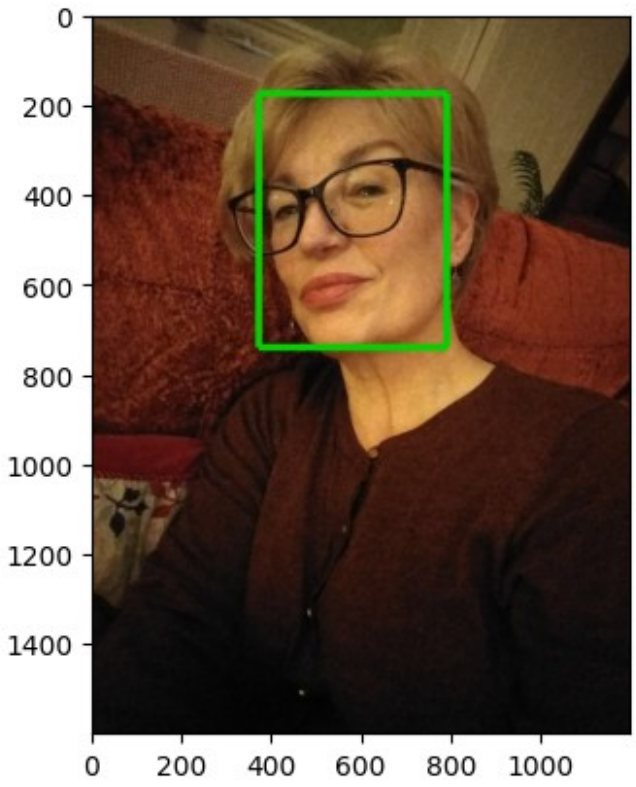
draw = ImageDraw.Draw(img)
for faceRecord in response['FaceRecords']:
    box = faceRecord['Face']['BoundingBox']
    left = imgWidth * box['Left']
    top = imgHeight * box['Top']
    width = imgWidth * box['Width']
    height = imgHeight * box['Height']

    points = ((left,top),(left+width,top),(left+width,top+height),
(left,top+height),(left,top))

    draw.line(points,fill='#00d400', width=15)

plt.imshow(img)
```

```
<matplotlib.image.AxesImage at 0x7f5f8290e9e0>
```



Task 5: Listing faces in the collection

Examine the image that you have in the collection.

```
maxResults=2
faces_count=0
tokens=True

response=client.list_faces(CollectionId=collection_id,
                           MaxResults=maxResults)

print('Faces in collection ' + collection_id)

while tokens:

    faces=response['Faces']

    for face in faces:
        print (face)
        faces_count+=1
    if 'NextToken' in response:
        nextToken=response['NextToken']
        response=client.list_faces(CollectionId=collection_id,

NextToken=nextToken,MaxResults=maxResults)
```

```
    else:
        tokens=False
```

```
Faces in collection Collection
{'FaceId': '51c54066-f5ea-4cff-ac3e-441e915275ff', 'BoundingBox':
{'Width': 0.34836500883102417, 'Height': 0.3531210124492645, 'Left':
0.30951398611068726, 'Top': 0.11020100116729736}, 'ImageId':
'59d87d59-b733-306a-94e8-0670b0a8dcdd', 'ExternalImageId': 'mum.jpg',
'Confidence': 99.99430084228516, 'IndexFacesModelVersion': '7.0'}
```

Task 6: Finding a face by using the collection

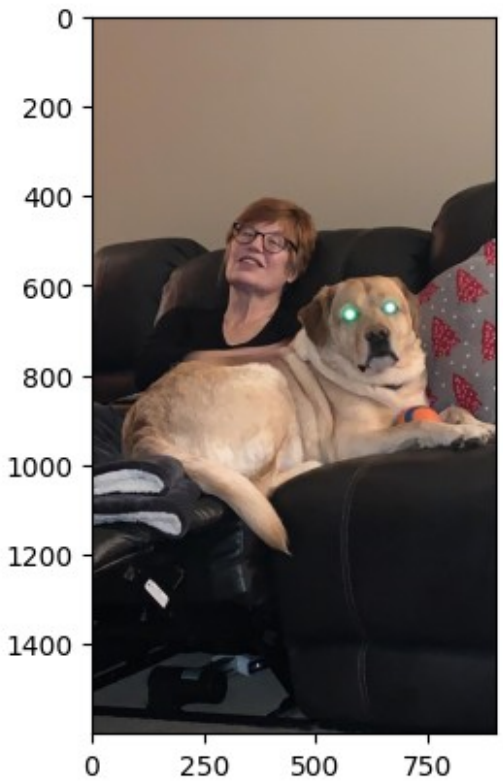
In this step, you will use the collection to detect a face in an image.

Use the provided sample image that's named *target.jpg* and upload it to this notebook.

```
targetfilename = "target.jpg"

targetimage = Image.open(targetfilename)
plt.imshow(targetimage)
```

```
<matplotlib.image.AxesImage at 0x7f5f829c81f0>
```



Next, call the `search_faces_by_image` operation and see if you get a match.

```
threshold = 70
maxFaces=2

with open(targetfilename, 'rb') as timage:
    response2=client.search_faces_by_image(CollectionId=collection_id,
                            Image={'Bytes': timage.read()},
                            FaceMatchThreshold=threshold,
                            MaxFaces=maxFaces)

faceMatches=response2['FaceMatches']
print ('Matching faces')
for match in faceMatches:
        print ('FaceId:' + match['Face']['FaceId'])
        print ('Similarity: ' + "{:.2f}".format(match['Similarity']) +
"%")
        print ('ExternalImageId: ' + match['Face']['ExternalImageId'])
        print
```

```
Matching faces
FaceId:51c54066-f5ea-4cff-ac3e-441e915275ff
Similarity: 99.91%
ExternalImageId: mum.jpg
```

Task 7: Drawing a bounding box around the discovered face

Draw a bounding box around the discovered face.

```
imgWidth, imgHeight = targetimage.size

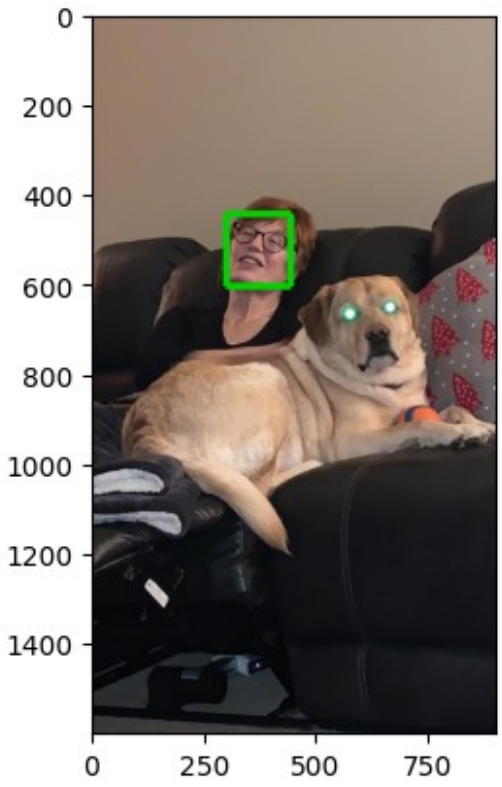
draw = ImageDraw.Draw(targetimage)

box = response2['SearchedFaceBoundingBox']
left = imgWidth * box['Left']
top = imgHeight * box['Top']
width = imgWidth * box['Width']
height = imgHeight * box['Height']

points = ((left,top),(left+width,top),(left+width,top+height),
(left,top+height),(left,top))
draw.line(points,fill='#00d400', width=15)

plt.imshow(targetimage)
```

```
<matplotlib.image.AxesImage at 0x7f5f82849c30>
```



Task 8: Deleting the collection

When you are finished, delete the collection. To do this, run the following code.

```
print('Attempting to delete collection ' + collection_id)
status_code=0
try:
    response=client.delete_collection(CollectionId=collection_id)
    status_code=response['StatusCode']
    print('All done!')
    print(status_code)

except ClientError as e:
    if e.response['Error']['Code'] == 'ResourceNotFoundException':
        print ('The collection ' + collection_id + ' was not found ')
    else:
        print ('Error other than Not Found occurred: ' +
e.response['Error']['Message'])
    status_code=e.response['ResponseMetadata']['HTTPStatusCode']


Attempting to delete collection Collection
All done!
200
```

Congratulations!

You have completed this lab, and you can now end the lab by following the lab guide instructions.